

LOOPING STATEMENTS

What are Looping Statements?

Looping statements are a fundamental part of programming that allow you to execute a block of code repeatedly. Think of it like a baker making cookies: instead of repeating the mixing, shaping, and baking steps manually for every single cookie, a loop automates the process for the entire batch. This makes your code more efficient, less repetitive, and easier to manage.

Python provides two primary types of loops: the `for` loop and the `while` loop.

The `for` Loop

The **`for` loop** is best used when you want to iterate over a known sequence of items, such as the elements in a list, the characters in a string, or a range of numbers. It's like a chef going through an ingredient list—the loop processes each item in the sequence one by one until it reaches the end.

Key Features of `for` Loops

- **Iterating over Sequences:** The most common use is to perform an action for each item in a list, tuple, or string.

```
Python
# Looping through a list of fruits
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"Current fruit: {fruit}")
```

- **The `range()` function:** This is a powerful tool for repeating an action a specific number of times. `range(stop)` generates numbers from 0 up to (but not including) `stop`. You can also specify a `start` and `step` value, like `range(start, stop, step)`.

```
Python
# Print numbers from 1 to 3
for i in range(1, 4):
    print(i) # Output: 1, 2, 3
```

- **The `enumerate()` function:** This handy function is used when you need both the **index** and the **value** of each item in a sequence during the loop.

```
Python
# Print each fruit with its index
for index, fruit in enumerate(fruits):
    print(f"Index {index}: {fruit}")
```

- **Nested `for` Loops:** You can place one `for` loop inside another. This is very useful for working with 2D data, like a grid or a matrix. For each single iteration of the outer loop, the inner loop will run to completion.

```
Python
# Print pairs of numbers
for i in range(2): # Outer loop runs twice (i=0, i=1)
    for j in range(3): # Inner loop runs three times for each outer loop
        print(f"({i}, {j})")
```

Shorthand `for` Loops

- **List Comprehension:** A concise and highly readable way to create a new list. It combines a `for` loop, an expression, and an optional `if` condition into a single, elegant line.

Python

Create a new list with the squares of numbers from 0 to 4

```
squares = [x**2 for x in range(5)]
```

```
print(squares) # Output: [0, 1, 4, 9, 16]
```

- **Generator Expressions:** These look just like list comprehensions but use parentheses `()`. The key difference is that they are "lazy"—they generate values one at a time and don't store the entire list in memory. This makes them extremely memory-efficient for very large datasets.

Python

Create a generator object for squares

```
squares_gen = (x**2 for x in range(5))
```

To see the values, you must iterate over it or convert it to a list

```
print(list(squares_gen)) # Output: [0, 1, 4, 9, 16]
```

The while Loop

The **while loop** is used to execute a block of code repeatedly **as long as a specified condition remains True**. It's the perfect choice when you don't know in advance how many times the loop needs to run. Think of it like waiting for a download to finish; you keep waiting *while* the progress is less than 100%.

Structure and Common Pitfalls

A `while` loop requires careful setup to avoid running forever (an **infinite loop**). There are three essential parts:

1. **Initialization:** A variable (often a counter) must be initialized *before* the loop begins.
2. **Condition:** The `while` statement checks this condition *before* each iteration.
3. **Update:** The variable must be changed *inside* the loop so that the condition will eventually become false, allowing the loop to end.

Python

A simple while loop that counts from 1 to 3

```
count = 1 # 1. Initialization
```

```
while count <= 3: # 2. Condition
```

```
    print(f"The count is {count}")
```

```
    count += 1 # 3. Update (crucial to avoid an infinite loop!)
```

```
print("Loop finished.")
```